QueryForge: Autonomous Text-to-SQL & Natural Language BI Engine

Document Version: 1.0.0
Architecture Tier: Standalone AI Microservice (Database-Agnostic / Guardrailed Agentic Execution)
Primary Stack: Python 3.12+ (FastAPI), SQLAlchemy 2.0 (Async), SQLGlot (AST Firewall), ChromaDB / pgvector (Schema RAG), Groq (LLaMA-3.3-70B) + Gemini 1.5 Flash, Pandas, Next.js 16 (AI Analytics Studio UI)

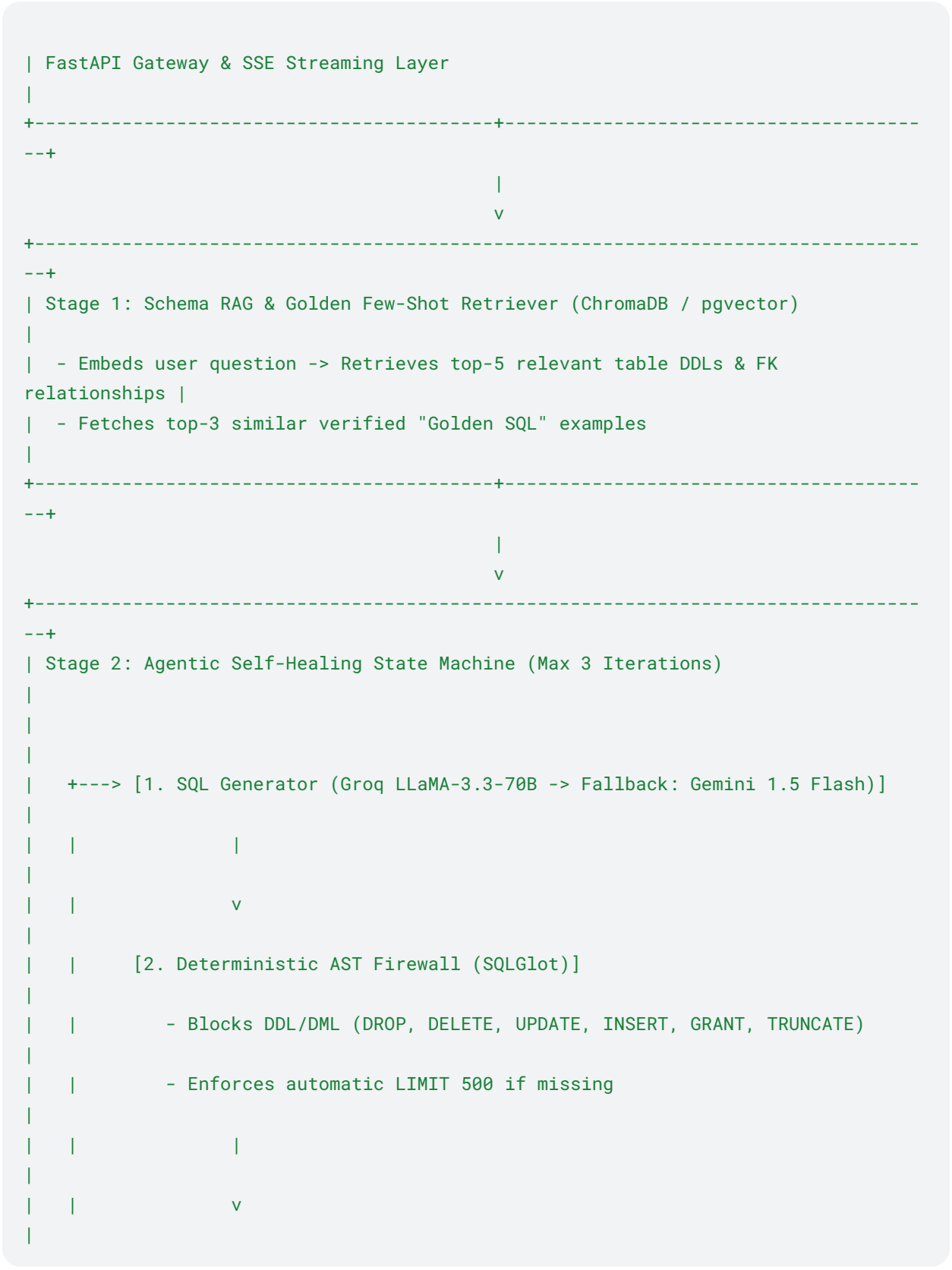
1. Executive Summary & Core Architecture

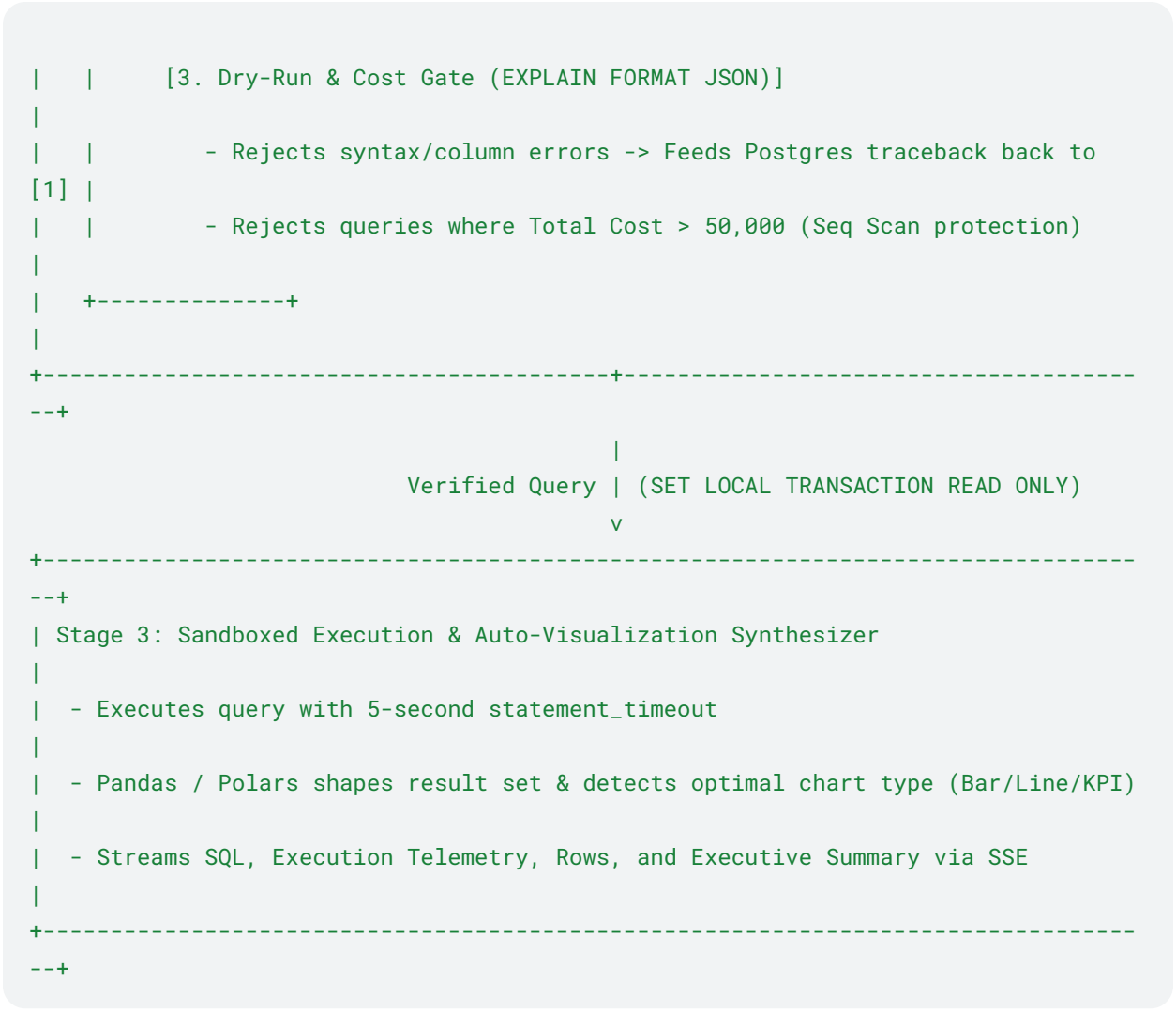
Standard Text-to-SQL wrappers fail in production for four reasons:

1. **Context Window Overflow:** Dumping a 150-table database DDL into every prompt wastes thousands of tokens and degrades LLM accuracy.
2. **Hallucinated Columns & Joins:** Models invent non-existent foreign keys or fail on complex PostgreSQL JSONB / window functions on the first attempt.
3. **Destructive & Unbounded Queries:** Without an AST firewall and query cost check, an LLM can execute `DROP TABLE` or run an unindexed `CROSS JOIN` that locks up the production database.
4. **Provider Rate Limits:** Relying on a single LLM API breaks the analytics pipeline during traffic spikes.

QueryForge solves all four by combining **Vector Schema RAG (ChromaDB / pgvector)**, a **Multi-Provider LLM Router (Groq LLaMA 3.3 + Gemini 1.5 Flash)**, **SQLGlot AST Validation**, **PostgreSQL `EXPLAIN (FORMAT JSON)` Cost Gating**, and a **3-Step Self-Healing Execution Loop**.




| FastAPI Gateway & SSE Streaming Layer
|
+----------+
--+
|
v
+----------
--+
| Stage 1: Schema RAG & Golden Few-Shot Retriever (ChromaDB / pgvector)
|
| - Embeds user question -> Retrieves top-5 relevant table DDLs & FK
relationships |
| - Fetches top-3 similar verified "Golden SQL" examples
|
+----------+
--+
|
v
+----------
--+
| Stage 2: Agentic Self-Healing State Machine (Max 3 Iterations)
|
|
|
| +---> [1. SQL Generator (Groq LLaMA-3.3-70B -> Fallback: Gemini 1.5 Flash)]
|
| | |
|
| | v
|
| | [2. Deterministic AST Firewall (SQLGlot)]
|
| | - Blocks DDL/DML (DROP, DELETE, UPDATE, INSERT, GRANT, TRUNCATE)
|
| | - Enforces automatic LIMIT 500 if missing
|
| | |
|
| | v
|



2. Core Engineering Modules

Module A: Dynamic Schema Introspection & Vector RAG (`app/core/schema_indexer.py`)

- Connects to any target database URL (`postgresql+asyncpg://...`) and queries `information_schema.tables`, `columns`, and `table_constraints`.
- Compiles each table into a semantic markdown card containing column types, foreign key links, and 3 sample distinct values for categorical columns.
- Indexes table cards and a curated set of **Golden Few-Shot Queries** into **ChromaDB** (or `pgvector`). When a user asks a question, only the top 5 relevant tables are injected into the prompt—cutting token usage by 85%.

Module B: Multi-Provider LLM Router (`app/core/llm_router.py`)

- Primary inference runs on **Groq (`llama-3.3-70b-versatile`)** for sub-300ms SQL generation.
- Automatic circuit-breaker fallback to **Google Gemini (`gemini-1.5-flash`)** if Groq returns `429 Too Many Requests`, `503`, or times out after 2.5 seconds.

Module C: SQLGlot AST Firewall & Read-Only Sandbox (`app/core/sql_guard.py`)

- Parses the generated SQL string into an Abstract Syntax Tree using `sqlglot`.
- **Hard Rules:**
    - Rejects any statement where the root AST node is not `exp.Select` or `exp.Union` (blocks `Update`, `Delete`, `Drop`, `Insert`, `Alter`, `Command`).
    - Blocks dangerous system functions (`pg_sleep`, `pg_read_file`, `dblink`).
    - Automatically injects `.limit(500)` into the AST if the query lacks a `LIMIT` clause to prevent memory exhaustion.

Module D: Self-Healing Execution Loop & Cost Gate (`app/core/agent_loop.py`)

- Opens a database transaction with `SET LOCAL TRANSACTION READ ONLY;` and `SET LOCAL statement_timeout = '5000';` (5 seconds).
- Runs `EXPLAIN (FORMAT JSON) <generated_sql>` first:
    - **Syntax / Column Error:** If PostgreSQL raises `UndefinedColumnError` or `GroupingError`, the agent catches the exact database error message, appends it to the conversation state, and asks the LLM to fix its mistake (up to 3 attempts).
    - **Cost Gate:** Parses `Plan -> Total Cost` from the JSON explain plan. If cost exceeds threshold, instructs the LLM to rewrite the query using indexed columns.

Module E: Auto-Chart & Insight Synthesizer (`app/core/chart_synthesizer.py`)

- Inspects the returned column types and cardinality (e.g., `1 datetime column + 1 numeric column` → `area_chart`; `1 categorical column + 1 numeric column` → `bar_chart`; `single scalar value` → `kpi_card`).
- Returns a Recharts-ready JSON specification alongside a 2-sentence executive summary of the data.

3. Database Schema (QueryForge Internal Metadata Store)

None

```
-- 1. Registered External Data Sources
CREATE TABLE data_sources (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(128) UNIQUE NOT NULL,
    dialect VARCHAR(32) NOT NULL DEFAULT 'postgresql',
    connection_uri_encrypted TEXT NOT NULL,
    tables_indexed INTEGER NOT NULL DEFAULT 0,
    last_synced_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 2. Golden Few-Shot SQL Examples (For RAG Accuracy)
CREATE TABLE golden_queries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    datasource_id UUID REFERENCES data_sources(id) ON DELETE CASCADE,
    natural_question TEXT NOT NULL,
    verified_sql TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 3. Query Execution & Self-Healing Audit Logs
CREATE TABLE query_audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    datasource_id UUID REFERENCES data_sources(id) ON DELETE SET NULL,
    user_question TEXT NOT NULL,
    generated_sql TEXT,
    self_heal_attempts SMALLINT NOT NULL DEFAULT 0,
    error_history JSONB NOT NULL DEFAULT '[]'::jsonb,
    explain_cost NUMERIC(12, 2),
    execution_time_ms NUMERIC(10, 2),
    row_count INTEGER,
    provider_used VARCHAR(32), -- 'GROQ' or 'GEMINI'
    status VARCHAR(24) NOT NULL CHECK (status IN ('SUCCESS', 'BLOCKED_AST',
'BLOCKED_COST', 'FAILED_EXECUTION')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_audit_status_created ON query_audit_logs (status, created_at
DESC);
```

4. REST API Specification

Method	Endpoint	Description
POST	/v1/datasources	Register a target database and trigger automatic schema introspection & ChromaDB indexing.
GET	/v1/datasources/{id}/schema	View indexed tables, foreign key graph, and sample DDL cards.
POST	/v1/query/ask	Submit a natural language question; runs the self-healing Text-to-SQL loop and returns data + chart config.
POST	/v1/query/stream	SSE streaming endpoint showing live agent steps: Retrieving Schema → Drafting SQL → AST Check → Self-Healing → Results.
POST	/v1/golden-queries	Save a verified question-SQL pair into the vector store to continuously improve accuracy.
GET	/v1/metrics/telemetry	Returns accuracy rate, first-shot pass rate vs. self-healed rate, average latency, and Groq/Gemini fallback split.

5. Standalone Production Folder Structure

```
None
query-forge/
├── docker-compose.yml             # QueryForge Postgres + ChromaDB + Target Demo E-Commerce DB
├── Dockerfile
├── pyproject.toml                 # fastapi, sqlalchemy, asyncpg, sqlglot, chromadb, httpx, pandas
├── app/
```

```
|   ├── main.py                    # FastAPI initialization & lifespan
|   ├── config.py                  # Pydantic Settings (Groq, Gemini, DB URLs,
Cost caps)
|   ├── database.py                # Internal metadata DB session + Dynamic
Target DB pool
|   ├── api/
|   |   └── v1/
|   |       ├── datasources.py     # DB registration & schema sync routes
|   |       ├── query.py           # /ask and /stream (SSE) Text-to-SQL endpoints
|   |       ├── golden.py          # Few-shot training pair CRUD
|   |       └── telemetry.py       # Agent performance & self-healing metrics
|   ├── core/
|   |   ├── llm_router.py          # Groq LLaMA-3.3-70B with automatic Gemini 1.5
Flash failover
|   |   ├── schema_indexer.py      # Introspects information_schema -> ChromaDB
embeddings
|   |   ├── sql_guard.py           # SQLGlot AST parser, read-only & LIMIT
enforcement
|   |   ├── agent_loop.py          # Stateful self-healing generator -> EXPLAIN
-> executor
|   |   └── chart_synthesizer.py   # Auto-detects Bar/Line/Pie/KPI config from
result columns
|   ├── models/                    # SQLAlchemy 2.0 models (DataSource,
GoldenQuery, QueryAuditLog)
|   └── schemas/                   # Pydantic v2 request/response contracts
├── scripts/
|   └── seed_demo_ecommerce_db.py  # Seeds a realistic 5-table SaaS/E-Commerce DB
(Orders, Users, Products, Refunds, Events)
├── tests/
|   ├── test_sql_guard.py          # Tests SQL injection & DDL blocking via
SQLGlot
|   └── test_self_healing.py       # Simulates broken column names and verifies
auto-repair
└── dashboard/                     # Next.js 16 Standalone AI Analytics Studio UI
```

6. Next.js 16 Standalone "AI Analytics Studio" UI

To make this an unforgettable live demo on your portfolio, the standalone Next.js 16 frontend features three panels:

1. **Natural Language Command Bar + Pre-built Demo Prompts:**

    - One-click buttons for recruiters:
        - `"Show monthly revenue growth and refund rate by country for the last 6 months"` (Multi-table `JOIN` + `DATE_TRUNC` aggregation).
        - `"Test Self-Healing: Query a non-existent column user_total_rev"` (Visually shows Attempt 1 failing with Postgres `UndefinedColumnError`, the agent reading the error, and Attempt 2 succeeding automatically).
        - `"Test Security Guardrail: DROP TABLE orders;"` (Visually triggers the red **SQLGlot AST Firewall** alert in <2ms).
2. **Live Agent Reasoning Trace (Step-by-Step Timeline):**
    - Displays the exact tables retrieved via Schema RAG, the SQLGlot AST verification badge, the PostgreSQL `EXPLAIN` cost score, and whether Groq or Gemini served the query.
3. **Dynamic Chart & Data Grid:**
    - Automatically renders a responsive Recharts Bar/Area chart, the formatted SQL with syntax highlighting, and a CSV export button.

7. 4-Phase Implementation Roadmap

- **Phase 1: Foundation & Demo E-Commerce Database (Days 1–2)**
  Set up `docker-compose.yml`, SQLAlchemy 2.0 models for QueryForge's metadata store, and run `seed_demo_ecommerce_db.py` to populate a realistic 5-table dataset (`customers`, `products`, `orders`, `order_items`, `refunds`) so queries return rich analytical data.
- **Phase 2: SQLGlot AST Firewall & Groq/Gemini LLM Router (Days 3–5)**
  Build `sql_guard.py` (blocking `DROP`/`UPDATE`/unbounded queries via AST) and `llm_router.py` with automatic Groq-to-Gemini fallback.
- **Phase 3: Schema RAG & Self-Healing Execution Loop (Days 6–9)**
  Build `schema_indexer.py` to introspect `information_schema` into ChromaDB, and implement `agent_loop.py` with `SET LOCAL TRANSACTION READ ONLY`, `EXPLAIN (FORMAT JSON)` cost checking, and the 3-attempt error-repair loop.
- **Phase 4: Next.js 16 Analytics Studio & Evaluation Suite (Days 10–12)**
  Build the Next.js 16 UI with live SSE reasoning steps, auto-chart rendering, and the 1-click "Simulate Self-Healing" and "Simulate SQL Injection Attack" demo buttons.